MODULE 15 · DOMAIN 3 - DATA LOADING, UNLOADING, AND CONNECTIVITY

Snowpipe, Streaming, streams, tasks and dynamic tables

The densest module in the course. Five mechanisms, and one question that separates them.

DOMAIN 3 of 5 WEIGHT 18% MODULE 15 of 31

admin@sthithapragna.com

Batch loading is a **statement you run**. Continuous ingestion is a **thing that runs itself**.

Snowpipe

- **Serverless.** Snowflake provides the compute. No warehouse of yours, and it bills as serverless credits.
- Two ways to trigger it: **auto-ingest** from cloud event notifications, or calls to the **Snowpipe REST API**.
- Target latency is roughly **one minute** after a file lands. It loads files, not rows.
- A **PIPE** object wraps a COPY statement. Snowpipe is essentially COPY that runs itself when files appear.

> ON THE EXAM
>
> Auto-ingest needs a notification integration, which is the third of the four integration types from module 5.

Snowpipe details that get tested

- **Load metadata lives on the PIPE, not the table**, and is retained **14 days**, not 64.
- **Recreating a pipe wipes its load history**, so files already loaded can be loaded again. A real duplication risk.
- **Snowpipe does not support PURGE.** Use REMOVE periodically, or cloud storage lifecycle rules.
- **ON_ERROR defaults to SKIP_FILE**, not ABORT_STATEMENT.
- **ALTER PIPE … REFRESH** picks up files staged in the **previous 7 days**. Older files need a COPY statement.

> ON THE EXAM
>
> Three different retention numbers now: 64 days for COPY load metadata, 14 for a pipe, 7 for REFRESH. Keep them apart.

Snowpipe Streaming has two architectures

CLASSIC

Channels open against the table

- SDK: snowflake-ingest-sdk, Java 8 or later
- Schema validation happens client-side
- Credit-based billing
- Supported now, deprecation planned

HIGH-PERFORMANCE

Channels open against a PIPE

- SDK: snowpipe-streaming, Java 11 or Python 3.9
- Schema validation is server-side, from the PIPE
- Throughput-based billing, up to 10 GB/s per table
- Latency of 5 to 10 seconds. ON_ERROR supports only CONTINUE

THE DISTINCTION THAT MATTERS

Streaming writes rows, not files. That is the whole point, and it is what separates it from Snowpipe.

Which architecture, and what is changing

- Classic is **fully supported today**, and Snowflake has stated it is **planned for deprecation**.
- A formal deprecation announcement is intended for **mid-2026**, followed by an **18-month migration window**.
- Snowflake's recommendation is to use the **high-performance architecture for new work**.
- The January 2026 study guide predates this split. **Expect questions about Snowpipe Streaming generally**, not about which architecture.

> ON THE EXAM
>
> Know that two exist and how they differ. Do not expect a question that turns on the SDK version.

Streams: an offset, not a copy

- A stream stores an **offset** in the source table's version history and returns rows changed since that point.
- **It does not store the data.** It is a bookmark, which is why creating one costs almost nothing.
- **Consuming a stream in a DML transaction advances the offset.** The change records are then gone from it.
- Two consumers sharing one stream will not both see the changes. **Create a separate stream per consumer.**
- **SYSTEM$STREAM_HAS_DATA** checks whether there is anything to process, which is how tasks skip empty runs.

> ON THE EXAM
>
> 'A bookmark, not a copy' explains the cost, the staleness behaviour, and why two consumers interfere.

The three stream types

TYPE	WHAT IT TRACKS
Standard	Inserts, updates and deletes.
Append-only	Inserts only. Updates, deletes and truncates are not recorded.
Insert-only	Inserts, on external tables only.

ON THE EXAM

An update appears as a DELETE and an INSERT pair, both with METADATA$ISUPDATE = TRUE. That is how you tell it apart from a real delete.

Where a stream can be created

- On **standard tables**, permanent or transient. Also on **directory tables** and **external tables**.
- On **views**, including secure and shared views, provided **change tracking is enabled** on the view and all underlying tables.
- Only the **object owner** can enable change tracking. Creating a stream on a view as the view owner enables it.
- The retention extension does **not** apply to directory tables or external tables, because they have no retention period.

> ON THE EXAM
>
> A stream on a view is the usual way to capture changes across a join without duplicating logic in every consumer.

Staleness, and the extension that saves you

- A stream goes **stale** when its offset falls outside the source table's retention. **Unconsumed changes are lost.**
- That is **data loss, not an error**. Drop and recreate, which resets the offset to now.
- If retention is **under 14 days and the stream is unconsumed**, Snowflake extends retention to the stream's offset.
- Bounded by **MAX_DATA_EXTENSION_TIME_IN_DAYS, default 14, regardless of edition**.
- Check **STALE_AFTER** with DESCRIBE STREAM, not the retention setting.

> ON THE EXAM
>
> A stream on a 1-day table typically survives about fourteen days unconsumed, not one. CREATE OR REPLACE TABLE makes it stale immediately.

Tasks: serverless or your warehouse

- **Omit the WAREHOUSE parameter and the task is serverless.** Name a warehouse and it is user-managed.
- WAREHOUSE and USER_TASK_MANAGED_INITIAL_WAREHOUSE_SIZE are **mutually exclusive**. Unset WAREHOUSE to convert to serverless.
- For serverless tasks, Snowflake sizes the compute, **learning the right size** from completed runs after the first few.
- Tasks are **created suspended**. They must be resumed before anything happens.
- **SUSPEND_TASK_AFTER_NUM_FAILURES** auto-suspends a task that keeps failing.

> ON THE EXAM
>
> 'Created suspended' catches people constantly. A perfectly correct task that nobody resumed does nothing at all.

Scheduling and task graphs

- Schedule with a **cron expression including a time zone**, or a fixed interval.
- **A schedule is required for a standalone task or the root task. A child task cannot have one** – it uses AFTER.
- With no schedule, a task runs only via **EXECUTE TASK**.
- A **task graph** is a DAG with exactly **one root task**. A **finalizer** runs at the end, set with FINALIZE.
- **OVERLAP_POLICY**: NO_OVERLAP, ALLOW_CHILD_OVERLAP, or ALLOW_ALL_OVERLAP. Under NO_OVERLAP an overrun **skips** the next slot.

> ON THE EXAM
>
> OVERLAP_POLICY replaces the deprecated ALLOW_OVERLAPPING_EXECUTION. Recognise both names.

Dynamic tables against streams and tasks

STREAM PLUS TASK

You write the logic

- Full control: MERGE, compound keys, custom rules
- You manage the schedule and the dependencies
- Sub-minute freshness is possible
- More moving parts to operate

DYNAMIC TABLE

Snowflake writes the logic

- You give a SELECT and a TARGET_LAG
- Snowflake infers dependencies and refresh order
- Minimum target lag of 1 minute
- Query definition is immutable: drop and recreate

A LIMIT WORTH REMEMBERING

Dynamic tables cannot be downstream of a materialized view, an external table, or a stream. That rules them out sometimes.

Monitoring things that run themselves

WHAT YOU WANT TO KNOW	WHERE TO LOOK
Is this pipe running, and is it behind?	SYSTEM$PIPE_STATUS
What has Snowpipe cost, and how many files?	PIPE_USAGE_HISTORY
Did these files load, and what went wrong?	COPY_HISTORY. Note that ABORT_STATEMENT runs never appear
Did my task run, and did it fail?	TASK_HISTORY. Set error notifications for failures
Is my stream about to go stale?	STALE_AFTER, via DESCRIBE STREAM or SHOW STREAMS
Is there anything to process right now?	SYSTEM$STREAM_HAS_DATA

ON THE EXAM

Continuous ingestion fails silently by design. Nobody is watching a screen, so these are how you find out.

Five mechanisms, one decision

THE REQUIREMENT	THE MECHANISM
Files land in a bucket and should load within about a minute	Snowpipe with auto-ingest
An application writes rows directly, seconds of latency	Snowpipe Streaming
A single scheduled bulk load, full control over timing	COPY INTO in a task
Process only what changed, with custom logic such as MERGE	A stream plus a task
Keep a derived result fresh, joins and aggregations, minutes of lag	A dynamic table
Always-current derived data, one table, no joins	A materialized view

ON THE EXAM

This extends the three-way tiebreaker from module 4 across the whole ingestion surface. Freshness and control decide it.

Openflow: an honest position

- **Openflow** is Snowflake's data integration service, built on **Apache NiFi**. Connectors move data in from external systems.
- Two deployment models: **Snowflake Deployments** on Container Services, and **BYOC** in your own cloud account.
- The **January 2026 study guide lists it but excludes it from scored questions** 'until it is globally GA'.
- Snowflake Deployments reached GA on **AWS, Azure and GCP in November 2025**, so that exclusion may already have lifted.
- **Know what it is and move on.** Check the current study guide PDF before you sit.

> ON THE EXAM
>
> This is the one genuinely uncertain area on the paper. Recognise the name, do not memorise connector details.

How this module gets tested

THE QUESTION SHAPE	THE ANSWER
'Seconds of latency, application writes rows'	Snowpipe Streaming. Snowpipe is about a minute
'Files loaded twice after a change to the pipe'	Recreating a pipe wipes its 14-day load history
'The stream shows nothing after a long weekend'	Check STALE_AFTER. Retention is extended up to 14 days
'Two processes read the same stream'	Consuming advances the offset. One stream per consumer
'The child task will not accept a schedule'	Only the root task has one. Children use AFTER
'Nothing happens after creating the task'	Tasks are created suspended. Resume it

ON THE EXAM

Six shapes covering the module. Four of the six are about behaviour when something goes wrong, not about configuration.

A stream sits on a table whose DATA_RETENTION_TIME_IN_DAYS is 1. The task that consumes it was suspended by accident on a Friday and nobody noticed until the following Thursday, six days later.

An engineer assumes the stream must be stale and the six days of changes lost. They prepare to drop and recreate it, plus run a manual backfill.

Is the engineer right?

A Yes. With 1-day retention the stream went stale on Saturday and the changes are unrecoverable.

B Yes, but only because the task was suspended. A running task would have kept the offset alive.

C Probably not. Retention is extended for an unconsumed stream up to MAX_DATA_EXTENSION_TIME_IN_DAYS, 14 by default. Check STALE_AFTER.

D No. Streams never go stale on permanent tables, because Fail-safe preserves the change records for 7 days.

◆ PAUSE AND DECIDE

Is the engineer right?

A. Stale on Saturday. Changes lost.

Reasons from the retention setting alone, which is exactly what the extension exists to prevent.

B. Only because the task was suspended.

Backwards. The extension applies precisely because the stream was unconsumed. A consuming task would have advanced the offset instead.

C. Extension up to 14 days. Check STALE_AFTER.

Documented: retention under 14 days is temporarily extended to the stream's offset, bounded by MAX_DATA_EXTENSION_TIME_IN_DAYS.

D. Fail-safe preserves change records.

Fail-safe is Snowflake-only disaster recovery. It has nothing to do with stream offsets.

THE SIGNAL

The extension is 14 days by default regardless of edition. Six days is comfortably inside it, and STALE_AFTER confirms.

A trading application produces individual events that must be queryable in Snowflake within about ten seconds. The application can call a Snowflake SDK directly. Volumes are high and steady.

An architect proposes having the application write small JSON files to a stage every few seconds, with Snowpipe auto-ingest loading them.

What is wrong with the proposal?

A. Snowpipe targets about a minute of latency and charges per file. Snowpipe Streaming writes rows directly and hits seconds.

B. Snowpipe cannot load JSON. The application must write Parquet for auto-ingest to work.

C. Nothing is wrong, provided ON_ERROR is set to CONTINUE on the pipe.

D. Auto-ingest requires a storage integration, which the application cannot create at runtime.

◆ PAUSE AND DECIDE

What is wrong with the proposal?

A Snowpipe is about a minute and per-file. Streaming writes rows.

Ten seconds is below Snowpipe's target, and tiny files every few seconds is the pattern the file-sizing guidance warns against.

B Snowpipe cannot load JSON.

It loads JSON perfectly well. Format is not the problem here.

C Nothing wrong with ON_ERROR = CONTINUE.

A sensible setting that does nothing about latency or per-file overhead.

D Auto-ingest needs a storage integration.

It needs a notification integration, created once by an administrator. Not something the application does at runtime.

THE SIGNAL

'The application can call an SDK directly' is the planted detail. It rules out the file-based route entirely.

> An engineer builds a three-stage pipeline as a task graph. The root task runs hourly on cron. They give each of the two child tasks a cron schedule as well, five and ten minutes past the hour, to stagger the work.
>
> The statements creating the child tasks fail.

Why, and how should the pipeline be built?

A Cron schedules cannot include minute precision. Use an interval schedule on the children instead.

B Each task in a graph needs its own warehouse. Assign warehouses and the schedules will be accepted.

C A task graph may contain at most two tasks. Split the pipeline into two graphs.

D Child tasks cannot have a schedule. Only the root task does; children are chained with AFTER.

◆ PAUSE AND DECIDE

Why, and how should the pipeline be built?

A Cron lacks minute precision.

Cron expressions include a minute field. Precision is not the problem.

B Each task needs its own warehouse.

Tasks can share a warehouse, or be serverless with no warehouse at all. Unrelated to the failure.

C At most two tasks per graph.

Invented. A task graph is a DAG with one root and as many children as the work needs.

D Only the root task has a schedule. Children use AFTER.

Documented directly, and it is also better design: children run when their predecessor finishes, not at a guessed clock time.

THE SIGNAL

Staggering by clock time is the instinct this rule removes. Dependency ordering is what a task graph exists to provide.

> A pipe has been auto-ingesting files for eight months. To change its COPY statement, an engineer runs CREATE OR REPLACE PIPE with the new definition, then resumes it.
>
> Within an hour, downstream reports show roughly doubled row counts for several recent days. The staged files were never removed, because Snowpipe does not support PURGE.

What happened?

A. The pipe's ON_ERROR reverted to CONTINUE on recreation, so previously rejected rows were loaded.

B. Recreating the pipe wiped its load history, so files still in the stage were treated as new and loaded again.

C. CREATE OR REPLACE PIPE triggers a full REFRESH of every file ever staged, regardless of age.

D. The target table's 64-day load metadata expired, allowing the reload.

◆ PAUSE AND DECIDE

What happened?

A ON_ERROR reverted to CONTINUE.

Would explain a few extra rows, not doubled counts across several days. Rejected rows are a different scale of problem.

B Recreating the pipe wiped its load history.

Load metadata lives on the pipe, not the table. Destroy the pipe and every staged file looks new.

C CREATE OR REPLACE triggers a full REFRESH.

REFRESH is a separate ALTER PIPE operation, and it only covers files staged in the previous 7 days.

D The table's 64-day metadata expired.

Snowpipe does not use the table's 64-day load metadata. Its history is on the pipe and retained 14 days.

THE SIGNAL

Two facts combine: history lives on the pipe, and PURGE is unsupported so the files were still there to reload.

What to carry into the next module

- **Snowpipe loads files, about a minute. Snowpipe Streaming writes rows, seconds.** That is the distinction that gets tested.
- **Pipe metadata is 14 days and lives on the pipe.** Recreating a pipe wipes it. No PURGE. ON_ERROR defaults to SKIP_FILE.
- **A stream is an offset, not a copy.** Consuming advances it, so use one stream per consumer.
- **Staleness is data loss**, but retention is extended up to MAX_DATA_EXTENSION_TIME_IN_DAYS, 14 by default. Check STALE_AFTER.
- **Only the root task has a schedule.** Children use AFTER. Tasks are created suspended.

> ON THE EXAM
>
> Next: connectors and integrations, then five scenarios closing Domain 3.